Sustavi baza podataka

Predavanja

5. Transakcija

travanj 2026.

Transakcija

- Izvršavanje programa koji pristupa bazi podataka radi obavljanja neke administrativne (poslovne) funkcije

  Bernstein, Newcomer: Principles of Transaction Processing

- Niz SQL naredbi koje mijenjaju podatke i koje se moraju izvršiti u cijelosti ili se (u cijelosti) ne smiju izvršiti

  IBM Informix Guide to SQL: Tutorial

- Logička jedinica posla (*logical unit of work*) koja sadrži jednu ili više SQL naredbi. Transakcija grupira SQL naredbe tako da su te naredbe potvrđene (primijenjene na bazi podataka), ili poništene.

  Oracle 11g Release 2 (11.2)

Definicija: Transakcija je niz logički povezanih operacija koje se izvršavaju kao cjelina i prevode bazu podataka iz jednog u drugo **konzistentno stanje***

* baza podataka je konzistentna ako zadovoljava sva definirana integritetska ograničenja

Transakcija

Primjer:

- zadani iznos sredstava prebaciti s prvog zadanog na drugi zadani račun (stanje prvog računa umanjiti, a drugog računa uvećati).

racun

brRac	stanje
101	1000.00
102	500.00
103	2500.00
104	200.00

Logički povezane operacije:

```
umanjiti stanje na prvom računu (SQL: UPDATE)
uvećati stanje na drugom računu (SQL: UPDATE)
```

Ili obje UPDATE operacije moraju biti obavljene, ili niti jedna!

Sintaksa - sa stanovišta korisnika

- transakcija se opisuje kao program ili odsječak programa, SPL, PL/SQL, Transact SQL, Java+JDBC, C+ODBC, itd, kojim se izvršavaju SQL naredbe:
  - **`SELECT, UPDATE, INSERT, DELETE, ...`**
  - **`BEGIN [WORK], COMMIT [WORK], ROLLBACK [WORK]`**

  - **`BEGIN [WORK]`** nije u skladu s ISO/ANSI SQL standardom
    - ISO/ANSI SQL: **`START TRANSACTION`**

- transakcije se ne mogu ugnježđivati - svaka korisnička sjednica (*user session*), može u jednom trenutku imati najviše jednu aktivnu transakciju

Sintaksa - sa stanovišta korisnika

Primjer:

- napisati programski kôd za transakciju kojom se zadani iznos sredstava prebacuje s prvog zadanog na drugi zadani račun (stanje prvog računa umanjiti, a drugog računa uvećati).

```
varijable:  brRac1   ← broj prvog računa
            brRac2   ← broj drugog računa
            iznos    ← iznos kojeg treba prebaciti
```

racun

brRac	stanje
101	1000.00
102	500.00
103	2500.00
104	200.00

Sintaksa - ako se koristi implicitni početak transakcije

Oracle (PL/SQL) ili IBM Informix (SPL)

- **implicitni početak transakcije:** početkom transakcije smatra se prva SQL naredba izvršena u okviru korisničke sjednice ili prva SQL naredba izvršena neposredno nakon završetka prethodne transakcije
- **kraj transakcije:** mora biti obilježen (zadan) eksplicitno, jednom od naredbi:
  - COMMIT [WORK] ili ROLLBACK [WORK] naredbe iz ISO/ANSI SQL standarda
  - ako korisnička sjednica sjednica bude prekinuta prije nego se obavi COMMIT ili ROLLBACK, SUBP će transakciju poništiti (kao da je obavljena naredba ROLLBACK)

Primjer:

```
-- započela je korisnička sjednica
--> ovdje se početak transakcije podrazumijeva
UPDATE racun SET stanje = stanje - iznos WHERE brRac = brRac1;
UPDATE racun SET stanje = stanje + iznos WHERE brRac = brRac2;
COMMIT WORK;  -- ili ROLLBACK WORK
--> ovdje se početak transakcije podrazumijeva
sljedeća SQL naredba;
```

Sintaksa - ako se koristi eksplicitni početak transakcije

IBM Informix (SPL) ili SQL Server (Transact SQL)

- **početak transakcije:** obilježava (zadaje) se eksplicitno
  - BEGIN [WORK] ili START TRANSACTION ili BEGIN TRANSACTION

- **kraj transakcije:** mora biti eksplicitno obilježen (zadan)
  - COMMIT [WORK] ili ROLLBACK [WORK]
  - ako korisnička sjednica sjednica bude prekinuta prije nego se obavi COMMIT ili ROLLBACK, SUBP će transakciju poništiti (kao da je obavljena naredba ROLLBACK)

Primjer:

```
BEGIN WORK;
UPDATE racun SET stanje = stanje - iznos WHERE brRac = brRac1;
UPDATE racun SET stanje = stanje + iznos WHERE brRac = brRac2;
COMMIT WORK;
```

Sintaksa - bez određivanja granica transakcija

- ako granice transakcija nisu određene (niti implicitnim početkom i eksplicitnim završetkom, niti eksplicitnim početkom i završetkom), smatra se da transakcija započinje neposredno prije početka, a završava neposredno nakon završetka svake zasebne SQL naredbe. Podrazumijeva se COMMIT ako je naredba uspješno završila, inače se podrazumijeva ROLLBACK.

Primjer:

```
   -- ovdje se podrazumijeva početak transakcije (kao da je zaista obavljen BEGIN)
UPDATE racun SET stanje = stanje - iznos WHERE brRac = brRac1;
   -- ovdje se podrazumijeva kraj transakcije i to:
         COMMIT ako je SQL naredba obavljena bez pogreške, ROLLBACK inače


   -- ovdje se podrazumijeva početak transakcije (kao da je zaista obavljen BEGIN)
UPDATE racun SET stanje = stanje + iznos WHERE brRac = brRac2;
   -- ovdje se podrazumijeva kraj transakcije i to:
         COMMIT ako je SQL naredba obavljena bez pogreške, ROLLBACK inače
```

Sintaksa - sa stanovišta korisnika

- `setAutoCommit(false)`: implicitni početak transakcije. Kraj transakcije i ovdje mora biti eksplicitno zadan

Java varijable:

brRac1	←	broj prvog računa
brRac2	←	broj drugog računa
iznos	←	iznos kojeg treba prebaciti
conn	←	*Connection* objekt

```
Statement stmt = conn.createStatement();
conn.setAutoCommit(false);

// transakcija započinje prvom izvršnom SQL naredbom
stmt.executeUpdate("UPDATE racun SET stanje = stanje - " + iznos +
                   "  WHERE brRac = " + brRac1 );
stmt.executeUpdate("UPDATE racun SET stanje = stanje + " + iznos +
                   "  WHERE brRac = " + brRac2 );
conn.commit();    // ili conn.rollback()

// pozivom sljedeće execute... metode implicitno će započeti sljedeća transakcija
...
```

Sintaksa - sa stanovišta korisnika

- `setAutoCommit(true)`: granice transakcija nisu određene. Svaka DML naredba predstavlja zasebnu transakciju. Sljedeći odsječak stoga ne garantira konzistentnost!

Java varijable:

- **brRac1** ← broj prvog računa
- **brRac2** ← broj drugog računa
- **iznos** ← iznos kojeg treba prebaciti
- **conn** ← *Connection* objekt

```
Statement stmt = conn.createStatement();
conn.setAutoCommit(true);

// ovdje se podrazumijeva BEGIN
stmt.executeUpdate("UPDATE racun SET stanje = stanje - " + iznos +
                   "  WHERE brRac = " + brRac1 );
// ovdje se podrazumijeva COMMIT/ROLLBACK

// ovdje se podrazumijeva BEGIN
stmt.executeUpdate("UPDATE racun SET stanje = stanje + " + iznos +
                   "  WHERE brRac = " + brRac2 );
// ovdje se podrazumijeva COMMIT/ROLLBACK
```

Sintaksa - sa stanovišta SUBP-a

- sa stanovišta SUBP-a, transakcija je niz operacija čitanja i/ili pisanja (u ili iz elemenata baze podataka) koje su omeđene transakcijskim operacijama
  - operacije čitanja ili pisanja (*database operations*)
    - **`read(x, p), write(y, p), r[x], w[y]`**
    - u varijablu p učitaj vrijednost elementa x, u element y upiši vrijednost varijable p
  - transakcijske operacije (*transaction operations*)
    - **`start:`** početak nove transakcije
    - **`commit:`** završetak transakcije uz trajnu pohranu svih efekata transakcije
    - **`abort:`** prekid transakcije (kojeg prati poništavanje svih efekata transakcije)

```
start;
read(x, p);
r ← p * 10;
write(x, r);
commit;
```

```
r[x], w[x], c
```

Model transakcije



ACID svojstva transakcije

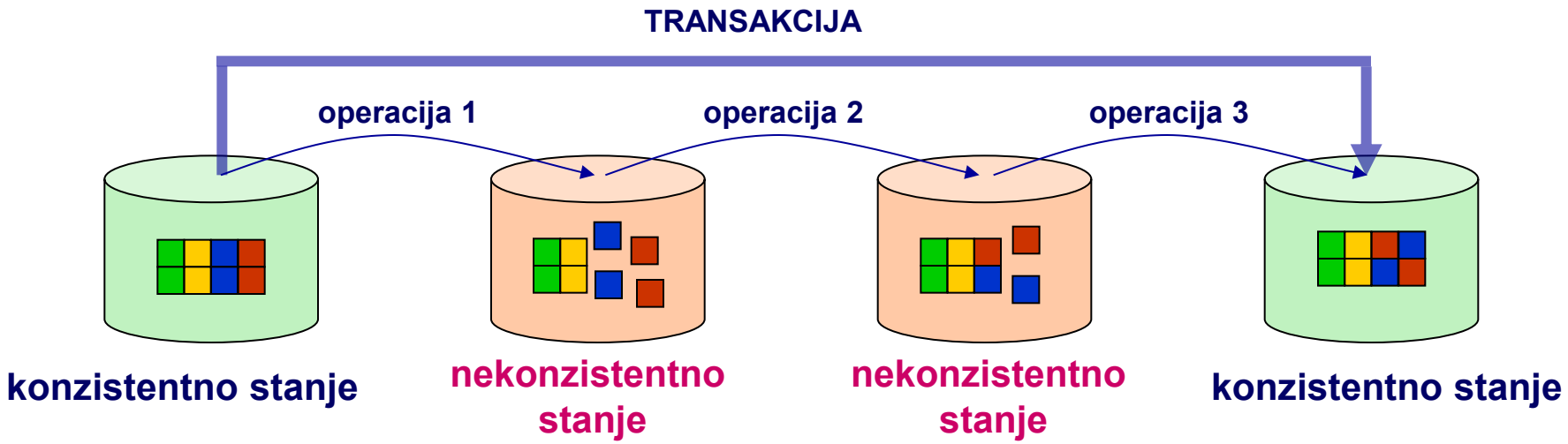
- SUBP mora osigurati sljedeća svojstva transakcija:

***A**tomicity*	Atomarnost
***C**onsistency*	Konzistentnost
***I**solation*	Izolacija
***D**urability*	Izdržljivost

Konzistentnost

- transakcija je korektna ako konzistentnu bazu podataka, u odsustvu drugih transakcija i pogrešaka*, prevodi iz jednog u drugo konzistentno stanje

  * pogreške će se detaljnije razmatrati u poglavlju o obnovi sustava



- **Konzistentnost (*Consistency*)** - transakcija zadovoljava svojstvo konzistentnosti ako je korektna

Konzistentnost

Primjer:

- prebaciti zadani **`iznos`** sredstava s računa **`brRac1`** na račun **`brRac2`**

- korektna transakcija

```
BEGIN WORK;
UPDATE racun SET stanje = stanje - iznos WHERE brRac = brRac1;
UPDATE racun SET stanje = stanje + iznos WHERE brRac = brRac2;
COMMIT WORK;
```

- nije korektna transakcija (posljedica npr. logičke pogreške u programu)

```
BEGIN WORK;
UPDATE racun SET stanje = stanje - iznos WHERE brRac = brRac1;
UPDATE racun SET stanje = stanje - iznos WHERE brRac = brRac2;
COMMIT WORK;
```

Konzistentnost

- na koji način osigurati svojstvo konzistentnosti?
- osigurati korektnost transakcije:
  - kad god je moguće koristiti eksplicitno opisana integritetska ograničenja
    - CHECK, PRIMARY, UNIQUE, FOREIGN KEY
    - okidači
    - pohranjene procedure
  - odgovornost programera
  - ili naprednog korisnika

- svojstvo konzistentnosti transakcije odnosi se samo na korektnost transakcije
  - dakle, pojam svojstva transakcije *konzistentnost* ne treba brkati s pojmom konzistentnosti baze podataka: konzistentnost baze podataka može (osim izvršavanjem transakcija koje nisu korektne), također biti narušena izvršavanjem transakcija koje ne zadovoljavaju svojstva atomarnosti, izolacije ili izdržljivosti

Izolacija

Primjer:

- početno stanje računa A je 1000 kn. Jedna iza druge se obavljaju dvije transakcije
- transakcija kojom se 600 kn isplaćuje s računa A

```
BEGIN WORK;
SELECT stanje INTO p1 FROM racun WHERE brRac = 'A';
LET p1 = p1 - 600;
UPDATE racun SET stanje = p1 WHERE brRac = 'A';
COMMIT WORK;
```

novo stanje na računu A: 400 kn

- transakcija kojom se 2000 kn uplaćuje na račun A

```
BEGIN WORK;
SELECT stanje INTO p2 FROM racun WHERE brRac = 'A';
LET p2 = p2 + 2000;
UPDATE racun SET stanje = p2 WHERE brRac = 'A';
COMMIT WORK;
```

novo stanje na računu A: 2400 kn

- transakcije su korektne - nakon njihovog obavljanja baza podataka je konzistentna

Izolacija

- ako SUBP operacije tih istih transakcija izvodi istodobno (naizmjence operacije jedne i druge transakcije u dvije korisničke sjednice)

```
BEGIN WORK;
SELECT stanje INTO p1 FROM racun WHERE brRac = 'A';
LET p1 = p1 - 600;
BEGIN WORK;
SELECT stanje INTO p2 FROM racun WHERE brRac = 'A';
LET p2 = p2 + 2000;
UPDATE racun SET stanje = p2 WHERE brRac = 'A';    -- stanje=3000
UPDATE racun SET stanje = p1 WHERE brRac = 'A';    -- stanje=400
COMMIT WORK;
COMMIT WORK;
```

novo stanje na računu A: 400 kn

- unatoč tome što su pojedinačne transakcije korektne, ako se njihove operacije obavljaju naizmjence, mogu narušiti konzistentnost baze podataka

- **Izolacija (*Isolation*)** - učinak transakcija koje se obavljaju istodobno (paralelno) mora biti jednak učinku tih istih transakcija koje bi se obavljale jedna iza druge (serijski)

Atomarnost

Primjer:

- prebaciti zadani **`iznos`** sredstava s računa **`brRac1`** na račun **`brRac2`**

```
BEGIN WORK;
UPDATE racun SET stanje = stanje - iznos WHERE brRac = brRac1;
UPDATE racun SET stanje = stanje + iznos WHERE brRac = brRac2;
COMMIT WORK;
```

Primijetiti: baza podataka u ovom trenutku nije konzistentna

- bez obzira na moguće događaje tijekom izvršavanja transakcije (nestanak napajanja, kvar računala, ...) dopušten je samo jedan od dva ishoda:
  - efekti svih naredbi transakcije su pohranjeni u bazi podataka
  - baza podataka je u stanju u kakvom je bila prije početka transakcije

- **Atomarnost (*Atomicity*) - ili** su efekti svih operacija transakcije pravilno pohranjeni u bazi podataka **ili** transakcija nije utjecala na stanje baze podataka
  - *all or nothing*

Atomarnost transakcije i interakcija s "vanjskim svijetom"

- budući da su operacije pisanja koje se obavljaju tijekom transakcije pod nadzorom SUBP-a, njihove efekte SUBP po potrebi može i poništiti (pojednostavljeno: vraćanjem starih vrijednosti elemenata). Kako to SUBP obavlja opisano je u sljedećim predavanjima
- problem nastaje kada se u okviru transakcije obavljaju i operacije koje utječu na okolinu SUBP-a (terminal ili slična U/I naprava). Tada svojstvo atomarnosti transakcije (za operacije izvršene izvan sustava) može biti dovedeno u pitanje

Atomarnost transakcije i interakcija s "vanjskim svijetom"

Primjer:

- što bi se moglo dogoditi kada bi bankomat (ATM) radio ovako?

```
BEGIN WORK
IF (SELECT stanje FROM racun
      WHERE brRac = p_brRac) >= p_iznos THEN
  UPDATE racun SET stanje = stanje - p_iznos
    WHERE brRac = p_brRac
  SYSTEM "prebroji_i_predaj " || iznos
  COMMIT WORK
ELSE
  ROLLBACK WORK
END IF
```

Za vježbu razmotrite kakve će biti posljedice kvara (npr. nestanka napajanja) koji bi se dogodio

- točno u trenutku prije obavljanja naredbe **`UPDATE`**?
- točno u trenutku nakon obavljanja naredbe **`UPDATE`**?
- točno u trenutku nakon što je korisniku predan novac (a prije **`COMMIT`**)?
- bi li pomoglo kad bi se operacija **`SYSTEM`** premjestila iza naredbe **`COMMIT`**?

Izdržljivost

- neka je sljedeća transakcija uspješno obavljena, tj. njezini efekti su u cijelosti pohranjeni u bazi podataka:

```
BEGIN WORK;
UPDATE racun SET stanje = stanje - iznos WHERE brRac = brRac1;
UPDATE racun SET stanje = stanje + iznos WHERE brRac = brRac2;
COMMIT WORK;
```

- ako se nakon obavljanja naredbe COMMIT dogodi kvar medija (*disk failure*) na kojem je pohranjena baza podataka, efekti potvrđene transakcije, koju korisnik smatra obavljenom, mogli bi biti izgubljeni

- **Izdržljivost (*Durability*)** - transakcija zadovoljava svojstvo izdržljivosti ako efekti transakcije, **nakon što je potvrđena**, neće biti izgubljeni neovisno o vrsti kvara koji bi se mogao dogoditi nakon što je transakcija potvrđena

Osiguravanje ACID svojstava transakcije

- Većina ACID svojstva osigurava se mehanizmima sustava za upravljanje bazama podataka
  - svojstva atomarnosti i izdržljivosti osigurava podsustav za obnovu
  - svojstvo izolacije osigurava podsustav za kontrolu istodobnog pristupa
- svojstvo konzistentnosti, odnosno korektnost transakcija, djelomično se može osigurati ugrađenim integritetskim ograničenjima, ali također ovisi i o ispravnosti programskog koda transakcije
  - svojstvo konzistentnosti se više neće razmatrati

Dodatak uz svojstvo konzistentnosti: odgođena provjera ugrađenih integritetskih ograničenja

- baza podataka može biti u nekonzistentnom stanju tijekom obavljanja transakcije - što se za to vrijeme dešava s ugrađenim integritetskim ograničenjima?
- provjera integritetskih ograničenja može se odgoditi do kraja transakcije SQL naredbama:

```
SET CONSTRAINTS ALL { IMMEDIATE | DEFERRED }
SET CONSTRAINTS constrName { IMMEDIATE | DEFERRED }
```

- **IMMEDIATE**: integritetska ograničenja provjeravaju se nakon svake izvedene DML naredbe (ne za vrijeme obavljanja DML naredbe!). Ako konačni rezultat DML naredbe narušava integritetska ograničenja, efekti DML naredbe se automatski poništavaju (*statement-level rollback*)
- **DEFERRED**: integritetska ograničenja se provjeravaju pri izvršavanju naredbe **COMMIT**
  - ako se utvrdi da je neko integritetsko ograničenje narušeno, transakcija se poništava

Dodatak uz svojstvo konzistentnosti: odgođena provjera ugrađenih integritetskih ograničenja

Primjer:

Svaki nastavnik predaje u točno jednoj grupi. Zamijeniti grupe za nastavnike 107 i 102

nastava

sifNast	sifGrupa
107	1.01
102	1.02
105	1.03

```
CREATE TABLE nastava (
   sifNast    INTEGER
 , sifGrupa   CHAR(4)
 , PRIMARY KEY (sifNast)
       CONSTRAINT pkNastava
 , UNIQUE (sifGrupa)
       CONSTRAINT uNastava
);
```

```
BEGIN WORK;
UPDATE nastava SET sifGrupa = '1.02'
   WHERE sifNast = 107; → POGREŠKA
UPDATE nastava SET sifGrupa = '1.01'
   WHERE sifNast = 102;
COMMIT WORK;
```

```
BEGIN WORK;
SET CONSTRAINTS uNastava DEFERRED;
UPDATE nastava SET sifGrupa = '1.02'
   WHERE sifNast = 107;
UPDATE nastava SET sifGrupa = '1.01'
   WHERE sifNast = 102;
COMMIT WORK;   → O.K.
```

Što će se dogoditi ako transakcija nije korektna?

```
BEGIN WORK;
SET CONSTRAINTS uNastava DEFERRED;
UPDATE nastava SET sifGrupa = '1.02'
   WHERE sifNast = 107;
COMMIT WORK;   → POGREŠKA i ROLLBACK
```

Prekid i poništavanje (*abort* i *rollback*) transakcije

- prekid transakcije iniciran od strane programa ili korisnika
  - u kontroliranim uvjetima utvrđeno je da transakcija ne može biti obavljena. Npr. u pohranjenoj proceduri je utvrđeno da bi daljnjim obavljanjem transakcije došlo do narušavanja integritetskih ograničenja, procedurom se obavlja naredba ROLLBACK*
  - abnormalni prekid programa (korisničke sjednice): SUBP nastavlja s radom ali poništava transakciju koja se izvršavala u toj korisničkoj sjednici
- prekid transakcije iniciran od strane sustava za upravljanje bazama podataka
  - abnormalni prekid rada SUBP-a (npr. nestanak napajanja). Sustav je prestao raditi, baza podataka je u nekonzistentnom stanju, ali će se efekti eventualno nezavršenih transakcija poništiti tijekom obnove, koja se obavlja prije nego sustav počne zaprimati nove transakcije.
  - sustav prekida i poništava jednu ili više transakcija tijekom razrješavanja potpunog zastoja

* SQL naredba ROLLBACK [WORK] predstavlja zahtjev za prekidom transakcije (*abort*) nakon kojeg SUBP odmah obavlja i poništavanje efekata transakcije (*rollback*).

Atomarnost, prekid i poništavanje na razini SQL naredbe

- neovisno o tome jesu li granice transakcije eksplicitno ili implicitno zadane, svaka pojedinačna SQL naredba se obavlja u cijelosti ili se njezini efekti u cijelosti poništavaju (ANSI Standard: atomarnost na razini SQL naredbe, *SQL-statement atomicity*)
- ako se SQL naredba ne može uspješno obaviti u cijelosti, obavlja se poništavanje na razini SQL naredbe (*statement-level rollback*)

Prekid i poništavanje na razini SQL naredbe
(statement-level rollback)

```
CREATE TABLE ispit (
   mbrSt   INTEGER
 , ocj     INTEGER
     CHECK (ocj BETWEEN 1 AND 5)
);
```



ispit

mbrSt	ocj
101	3
102	4
103	5

```
UPDATE ispit SET ocj = ocj + 1;
```

neke ocjene su možda bile promijenjene tijekom izvršavanja naredbe, ali u trenutku kada je naredba prekinuta zbog pogreške (narušavanja integritetskog ograničenja), svi njezini efekti su poništeni

```
BEGIN WORK;
... ostale SQL naredbe
UPDATE ispit SET ocj = ocj + 1;

... ostale SQL naredbe
COMMIT ili ROLLBACK WORK;
```

već u ovom trenutku poništeni su efekti naredbe UPDATE - sve su ocjene vraćene na početne vrijednosti. Efekti ostalih SQL naredbi iz ove transakcije ovog trenutka još nisu poništeni. Efekti ostalih SQL naredbi mogu se poništiti obavljanjem naredbe ROLLBACK

Subtransakcije

Ugnježđivanje transakcija nije dopušteno:

- pokušaj parcijalnog poništavanja transakcije na pogrešan način

```
BEGIN WORK;
SQL_1;
SQL_2;
BEGIN WORK;
  SQL_3;
  SQL_4;
  IF (uvjet_1) THEN
    ROLLBACK WORK;
  ELSE
    COMMIT WORK;
  END IF;
SQL_5;
COMMIT WORK;
```

- ispravan način za poništavanje samo određenih dijelova transakcije: *savepoints*

Savepoints

- naredba ROLLBACK poništava sve promjene u bazi podataka koje su transakcijom obavljene nakon početka transakcije

- u transakciji T je moguće označiti jednu ili više "točki pohrane" (*savepoints*)
- unutar transakcije T je moguće poništiti promjene koje su obavljene nakon bilo koje od označenih "kontrolnih" točaka (*savepoint*), a da se time ne prekine transakcija niti ponište efekti ostalih obavljenih DML naredbi

- SQL naredbe
  - **`SAVEPOINT savepoint_id;`**
  - **`ROLLBACK TO SAVEPOINT savepoint_id;`**

Savepoints

Primjer:

```
BEGIN WORK;
SQL_1;
SQL_2;
BEGIN WORK;
  SQL_3;
  SQL_4;
  IF (uvjet_1) THEN
    ROLLBACK WORK;
  ELSE
    COMMIT WORK;
  END IF;
SQL_5;
COMMIT WORK;
```



```
BEGIN WORK;
SQL_1;
SQL_2;
SAVEPOINT dioTrans;
  SQL_3;
  SQL_4;
  IF (uvjet_1) THEN
    ROLLBACK TO SAVEPOINT dioTrans;
  ELSE
    -- ništa
  END IF;
SQL_5;
COMMIT WORK;
```

Napomena: ROLLBACK TO SAVEPOINT nije naredba goto - ne vraća kontrolu toka programa na mjesto točke pohrane, nego samo poništava efekte naredbi obavljenih nakon točke pohrane

Klasifikacija transakcija

- ***on-line (short-life)* transakcije**: obavljaju se u relativno kratkom vremenu (tipično ne više od sekunde) i pristupaju malom broju elemenata baze podataka
- ***batch (long-life)* transakcije**: trajanje takvih transakcija je izraženo u minutama, praćeno pristupanjem velikom broju elemenata baze podataka
- **konverzacijske ili interaktivne transakcije**: za vrijeme obavljanja transakcije postoji interakcija s korisnikom

Model transakcije

Pojmovi i oznake

- transakcije se označavaju oznakama T_1, T_2, ... T_i, T_j, ...
- baza podataka se promatra kao statički skup (u smislu nepromjenjivosti kardinalnog broja skupa) imenovanih elemenata ili objekata
- kao element baze podataka može se promatrati dio n-torke (*field*), n-torka (*record*), fizička stranica (*page*, *block*), relacija, baza podataka. Veličina promatranog elementa određena je granulacijom (*granularity*). U većem dijelu razmatranja koje slijedi granulacija je irelevantna
- kao oznake elemenata baze podataka koriste se mala slova x, y, z, itd.
- sve operacije nad elementima baze podataka (*database operations*) odnose se na dvije osnovne vrste operacija: operaciju čitanja (*read*) i operaciju izmjene, odnosno pisanja (*write*). Navodi se vrsta operacije i naziv objekta. Vrijednost elementa je irelevantna
- operacije koje transakcija s oznakom T_i obavlja nad elementom x baze podataka označavat će se općenito $o_i[x]$, a konkretno za operacije čitanja i pisanja, koristit će se oznake $r_i[x]$ i $w_i[x]$
- transakcijske operacije (*transaction operations*) koje obavlja transakcija s oznakom T_i označavaju se a_i (*abort*) i c_i (*commit*)

Parcijalni poredak

- Parcijalni poredak*
  - Parcijalni poredak (*partial order*) L je uređeni par $(\Sigma, <)$. Skup Σ je domena parcijalnog poretka; relacija $<$ je relacija poretka. Relacija poretka je irefleksivna, asimetrična i tranzitivna relacija na skupu Σ. Ako za elemente $a, b \in \Sigma$ vrijedi $a < b$, odnosno uređeni par $(a, b) \in <$, tada se kaže da a prethodi b, odnosno da b slijedi nakon a. Ako ne vrijedi $a < b$ niti vrijedi $b < a$, tada su a i b neusporedivi (neporedani).

    *formalno ispravnije - *striktni* parcijalni poredak

    - Relacija ρ nad skupom P je:
      - irefleksivna ako $(\forall a \in P) \neg (a \rho a)$
      - asimetrična ako $(\forall a, b \in P) (a \rho b \Rightarrow \neg (b \rho a))$
      - tranzitivna ako $(\forall a, b, c \in P) (a \rho b \wedge b \rho c \Rightarrow a \rho c)$

Primjer:

- Domena $\Sigma = \{ a, b, c, d, e \}$
- Relacija poretka $< = \{ (a, c), (a, d), (a, e), (b, c), (b, d), (b, e), (c, d), (c, e) \}$
- Parcijalni poredak $L = (\Sigma, <)$

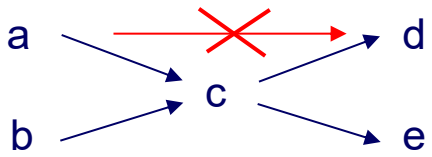
Parcijalni poredak i usmjereni graf

- Usmjereni graf i usmjereni aciklički graf
  - Usmjereni graf G = (N, E) sastoji se od skupa elemenata N koji se nazivaju čvorovi, te skupa E uređenih parova koji se nazivaju lúkovi. Lûk između elemenata a i b označavat će se s (a, b). Usmjereni aciklički graf je usmjereni graf koji ne sadrži petlje.

    - Parcijalni poredak L = (Σ, $<$) se može prikazati kao usmjereni aciklički graf G = (N, E) pri čemu je N = Σ, dok je E skup lúkova (a, b), a, b $\in \Sigma$, takvih da je a $<$ b.

Primjer:

- usmjereni aciklički graf za prikaz parcijalnog poretka iz prethodnog primjera:



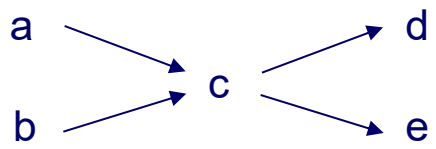
- lúkove implicirane svojstvom tranzitivnosti nije potrebno crtati

Topološki poredak usmjerenog acikličkog grafa

- Topološki poredak
  - Topološki poredak usmjerenog acikličkog grafa G = (N, E) je niz sastavljen od svih čvorova iz N poredanih tako da se element a nalazi u nizu ispred elementa b samo ako u G ne postoji put od čvora b do čvora a.

Primjer:

- prikaz mogućih topoloških poredaka za usmjereni aciklički graf iz prethodnog primjera:
  - a, b, c, d, e
  - b, a, c, d, e
  - a, b, c, e, d
  - b, a, c, e, d



- jednostavan postupak: u topološki poredak se redom mogu dodavati oni čvorovi iz grafa koji nemaju prethodnike među čvorovima koji se tog trenutka još uvijek ne nalaze u topološkom poretku.

Model transakcije

- Transakcija se modelira kao (striktni) parcijalni poredak
- Neka je Σ_i skup operacija transakcije T_i, $<_i$ je relacija poretka koja određuje međusobni poredak operacija iz Σ_i. Transakcija $T_i = (\Sigma_i, <_i)$ pri čemu vrijedi:

1. $\Sigma_i \subseteq \{ r_i[x], w_i[x] \mid x \text{ je objekt u bazi podataka} \} \cup \{ a_i, c_i \}$
2. $a_i \in \Sigma_i$ ako i samo ako $c_i \notin \Sigma_i$
3. ako je operacija $t \in \{ a_i, c_i \}$, tada za sve operacije p, $p \in \Sigma_i \wedge p \neq t$ vrijedi $p <_i t$
4. za svake dvije operacije $r_i[x], w_i[x] \in \Sigma_i$, vrijedi ili $r_i[x] <_i w_i[x]$ ili $w_i[x] <_i r_i[x]$

Objašnjenje:

1: definira koje su operacije transakcije dopuštene

2: u transakciji se nalazi jedna i samo jedna od operacija *abort* ili *commit*

3: operacija *abort* ili *commit* mora biti posljednja operacija svake transakcije

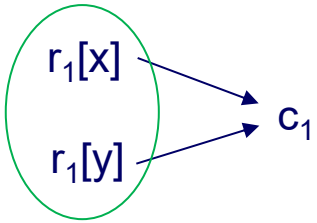
4: ako transakcija obavlja operaciju čitanja i operaciju pisanja nad istim elementom x, redoslijed obavljanja tih operacija utječe na rezultat operacije čitanja. Svojstvom se garantira da je za operaciju pisanja elementa x određen poredak u odnosu na operaciju čitanja istog elementa x

Parcijalni poredak, graf transakcije

Primjer:

- $T_1 = (\Sigma_1, <_1)$
  - $\Sigma_1 = \{ r_1[x], r_1[y], c_1 \}$
  - $<_1 = \{ (r_1[x], c_1), (r_1[y], c_1) \}$

- transakcija T_1, odnosno parcijalni poredak $(\Sigma_1, <_1)$ također se može prikazati sljedećim usmjerenim acikličkim grafom:

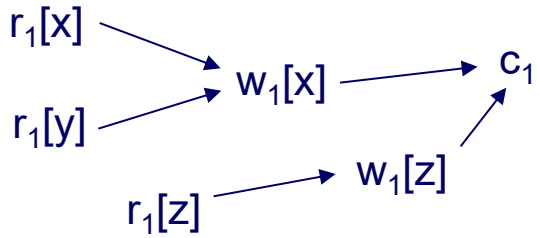


- operacije $r_1[x]$ i $r_1[y]$ su neusporedivi elementi transakcije - nije utvrđen njihov međusobni poredak
- operacija c_1 se mora dogoditi nakon $r_1[x]$ i $r_1[y]$

Parcijalni poredak, graf transakcije

Primjer:

- $T_1 = (\Sigma_1, <_1)$
  - $\Sigma_1 = \{ r_1[x], r_1[y], w_1[x], r_1[z], w_1[z], c_1 \}$
  - $<_1 = \{ (r_1[x], w_1[x]), (r_1[y], w_1[x]), (r_1[z], w_1[z]), (r_1[x], c_1), (r_1[y], c_1), (w_1[x], c_1), (r_1[z], c_1), (w_1[z], c_1) \}$
- transakcija T_1, odnosno parcijalni poredak $(\Sigma_1, <_1)$ također se može prikazati sljedećim usmjerenim acikličkim grafom:



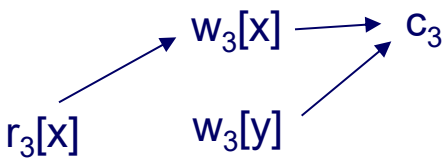
- $w_1[x]$ se mora dogoditi nakon $r_1[x]$ i $r_1[y]$
- nije određeno kojim redoslijedom se moraju obaviti npr. $r_1[x]$, $r_1[y]$ i $r_1[z]$.
- iako u relaciji poretka postoji par $(r_1[x], c_1)$, u grafu taj lûk nije potrebno crtati jer iz $r_1[x] < w_1[x]$ i $w_1[x] < c_1$, temeljem svojstva tranzitivnosti proizlazi $r_1[x] < c_1$.

Parcijalni poredak, graf transakcije

Dodatna razmatranja:

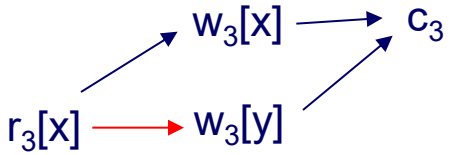
- osim lúkova koje graf **mora** sadržavati prema definiciji modela transakcije (sve operacije čitanja i pisanja nad istim elementom moraju biti poredane, sve operacije čitanja i pisanja prethode operacijama *commit*/*abort*), u graf se moraju ucrtati i lúkovi koji proizlaze iz semantike transakcije

Primjer:



vrijednost koju piše operacija $w_3[y]$ ne ovisi o rezultatu operacije $r_3[x]$

```
read(x, p);
display(p);
write(x, 35);
write(y, 27);
commit;
```



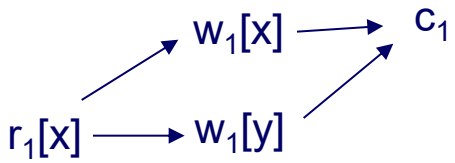
ako transakcija novu vrijednost za y određuje na temelju pročitane vrijednosti elementa x, dodaje se lûk između operacija $r_3[x]$ i $w_3[y]$

```
read(x, p);
write(x, 35);
p ← p + 10;
write(y, p);
commit;
```

Topološki poredak

Primjer:

- Transakcija T_1 je prikazana sljedećim usmjerenim acikličkim grafom



- Topološkim poretkom čvorova grafa može se dobiti prikaz transakcije u obliku niza operacija
  - $r_1[x], w_1[y], w_1[x], c_1$

  ili
  - $r_1[x], w_1[x], w_1[y], c_1$

Graf transakcije

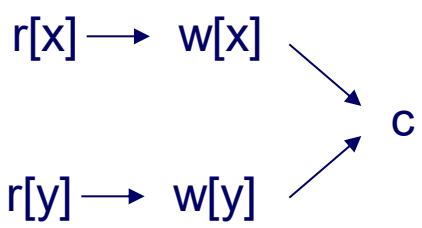
Primjer: zadani iznos sredstava prebaciti sa zadanog prvog na zadani drugi račun (stanje prvog računa umanjiti, a drugog računa uvećati).

x y

```
START TRANSACTION;
SELECT stanje INTO p FROM racun WHERE brRac = zadBrRac1;
SELECT stanje INTO q FROM racun WHERE brRac = zadBrRac2;
LET r = p - zadIznos;
LET s = q + zadIznos;
UPDATE racun SET stanje = r WHERE brRac = zadBrRac1;
UPDATE racun SET stanje = s WHERE brRac = zadBrRac2;
COMMIT;
```

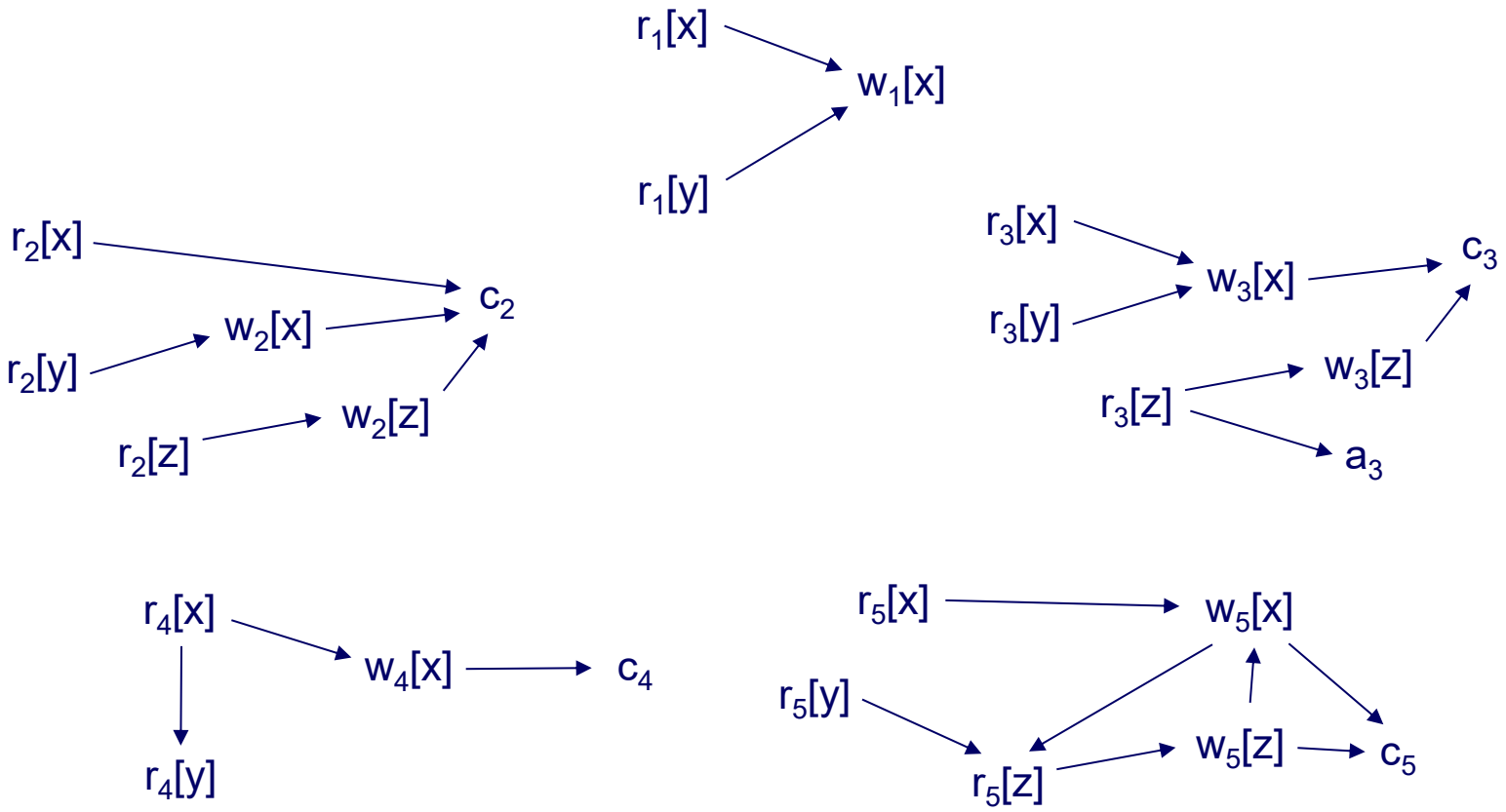
```
start;
read(x, p);
read(y, q);
-- izracunaj r, s
write(x, r);
write(y, s);
commit;
```

Model transakcije



Zadatak za vježbu

- za svaki graf utvrditi zašto se ne radi o ispravnom grafu transakcije



- rješenje zadatka se može provjeriti na sljedećoj stranici

Rješenje

1. u grafu ne postoji niti operacija a_1 niti operacija c_1
2. operacije $r_2[x]$ i $w_2[x]$ su neusporedive (neporedane), odnosno nije utvrđen poredak između $r_2[x]$ i $w_2[x]$
3. graf istovremeno sadrži operacije a_3 i c_3. Dodatno, a_3 nije poredana u odnosu na sve ostale operacije transakcije.
4. operacija c_4 nije poredana u odnosu na sve ostale operacije transakcije
5. graf nije acikličan (postoji ciklus $w_5[x] \rightarrow r_5[z] \rightarrow w_5[z] \rightarrow w_5[x]$), odnosno relacija poretka nije asimetrična (npr. vrijedi $w_5[x] < w_5[z]$ i $w_5[z] < w_5[x]$). Uočiti da zbog toga na temelju ovog grafa nije moguće konstruirati ispravan topološki poredak operacija

Literatura:

- P. Bernstein, V. Hadzilacos, N. Goodman. Concurrency control and recovery in database systems. Addison-Wesley Publishing Company. Reading, Massachusetts. 1987.
- H. Garcia-Molina, J. Ullman, J. Widom. Database system implementation. Prentice-Hall International, Inc. Upper Saddle River, New Jersey. 2000.
- R. Elmasri, S. Navathe. Fundamentals of database systems, 3rd ed. Addison-Wesley Publishing Company. Reading, Massachusetts. 2000.
- A. Silberschatz, H. Korth, S. Sudarshan. Database system concepts, 6th ed. McGraw-Hill. 2010.